FLOWCHARTS

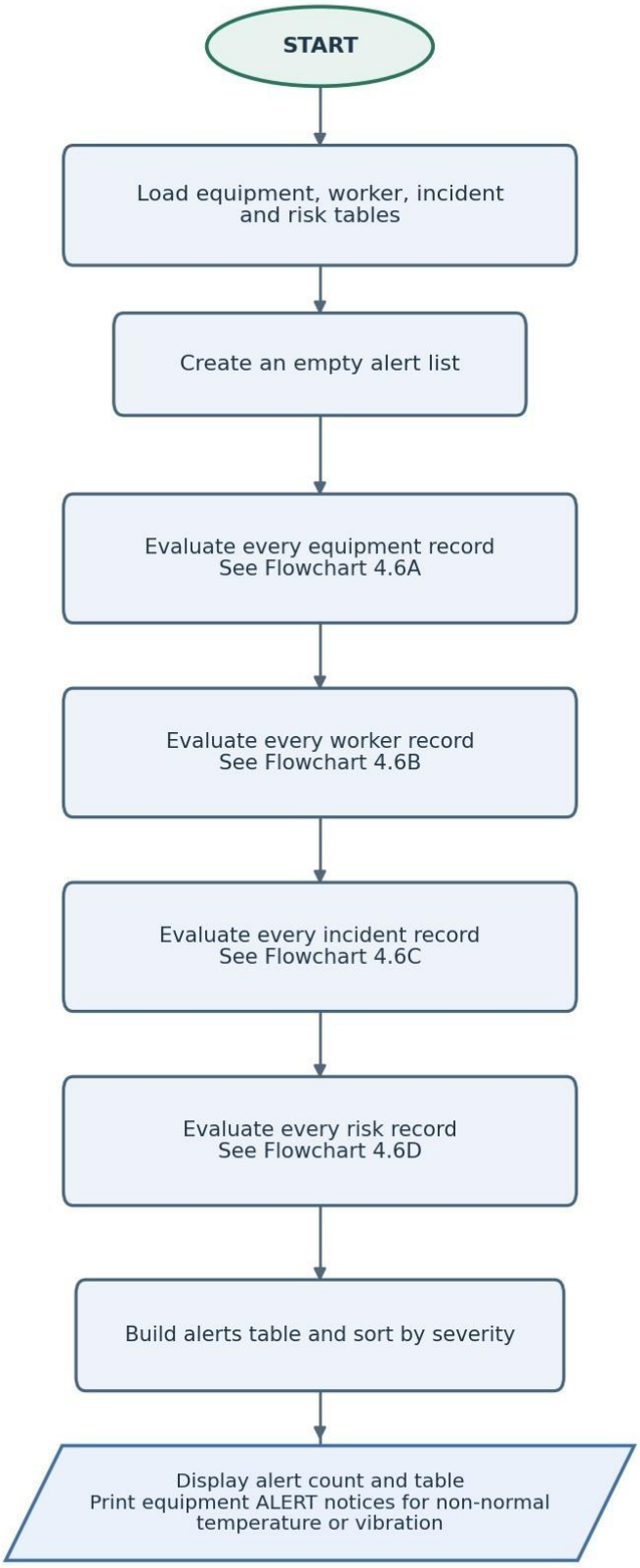
Alerts, Dashboard, Data Analysis and Reports

Flowcharts 4.6, 4.7a–4.7c and 4.8

Shape	Meaning
Oval	Start or end
Rectangle	Process or calculation
Diamond	Decision
Parallelogram	Input or output
Off-page connector	Continue in another flowchart

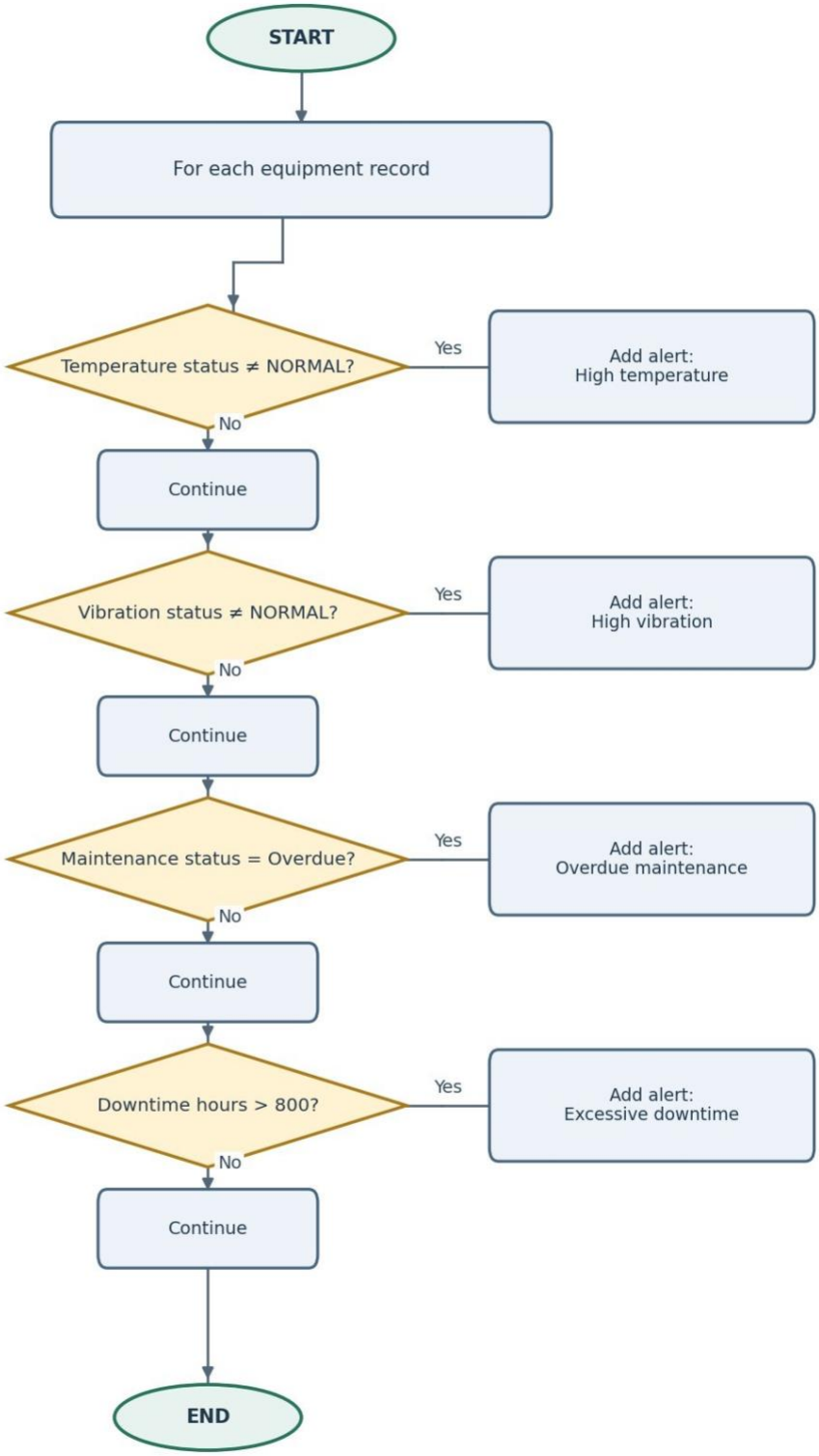
FLOWCHART 4.6 — ALERTS & NOTIFICATIONS

Overview of alert generation, severity sorting and display



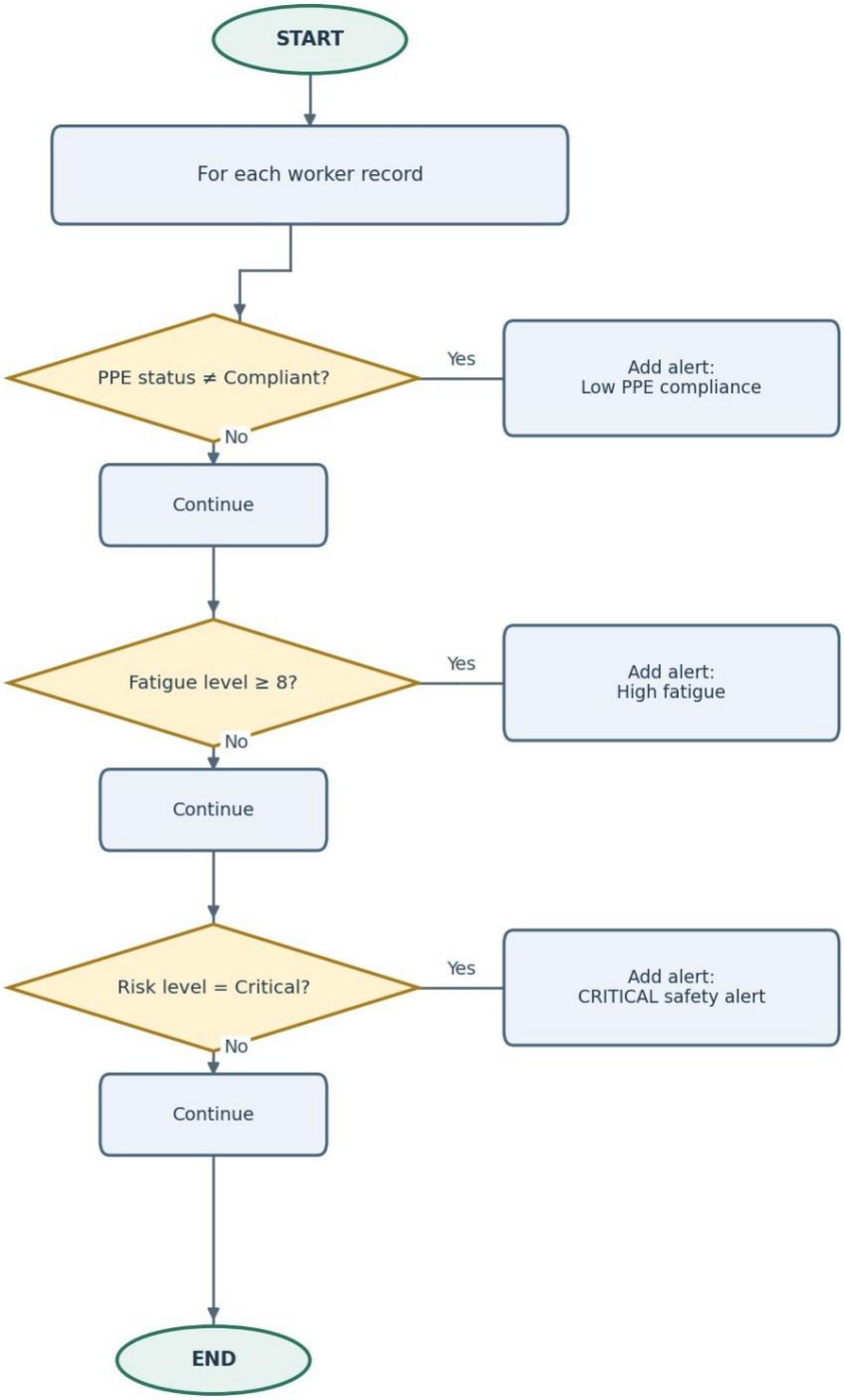
FLOWCHART 4.6A — EQUIPMENT ALERT RULES

For each equipment record



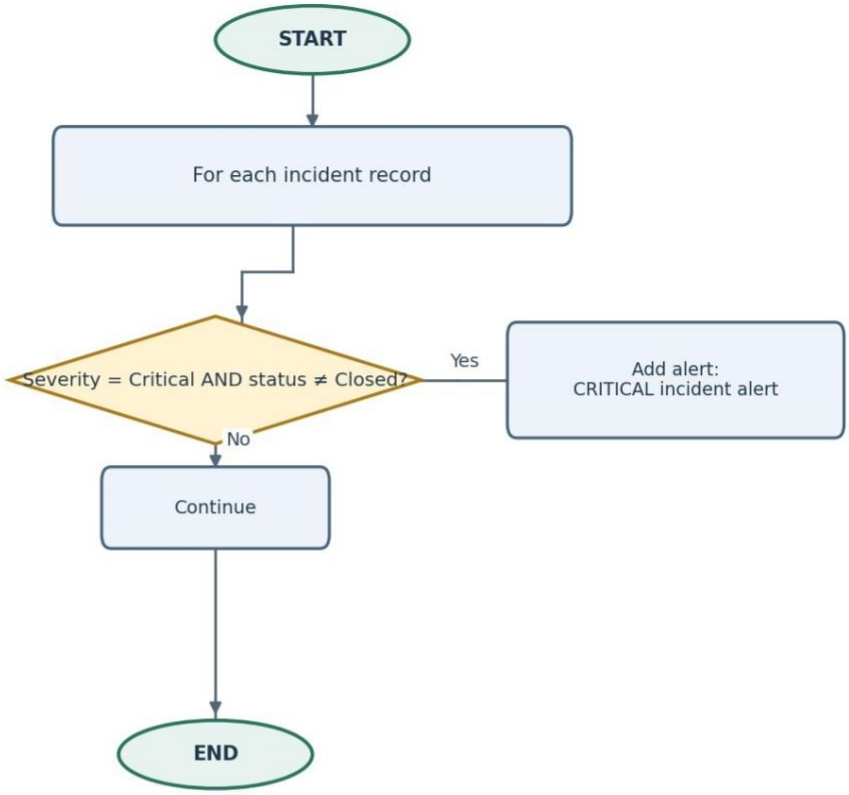
FLOWCHART 4.6B — WORKER ALERT RULES

For each worker record



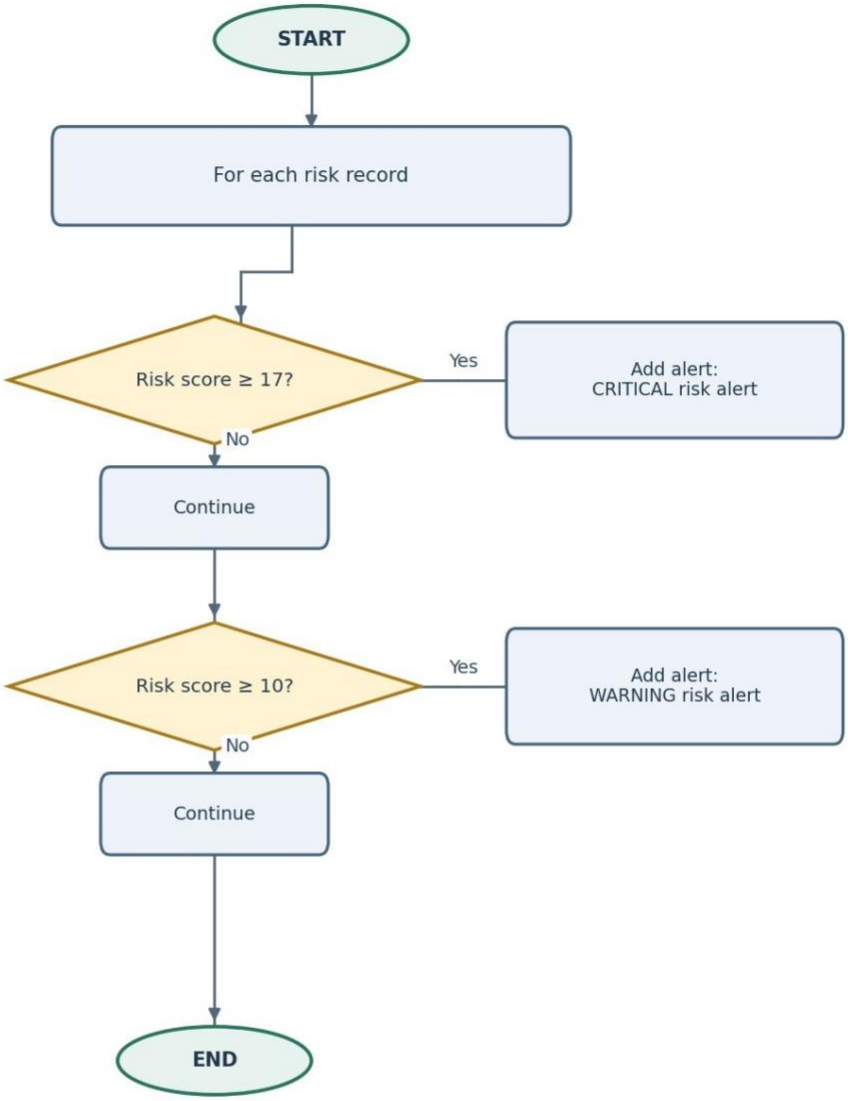
FLOWCHART 4.6C — INCIDENT ALERT RULES

For each incident record



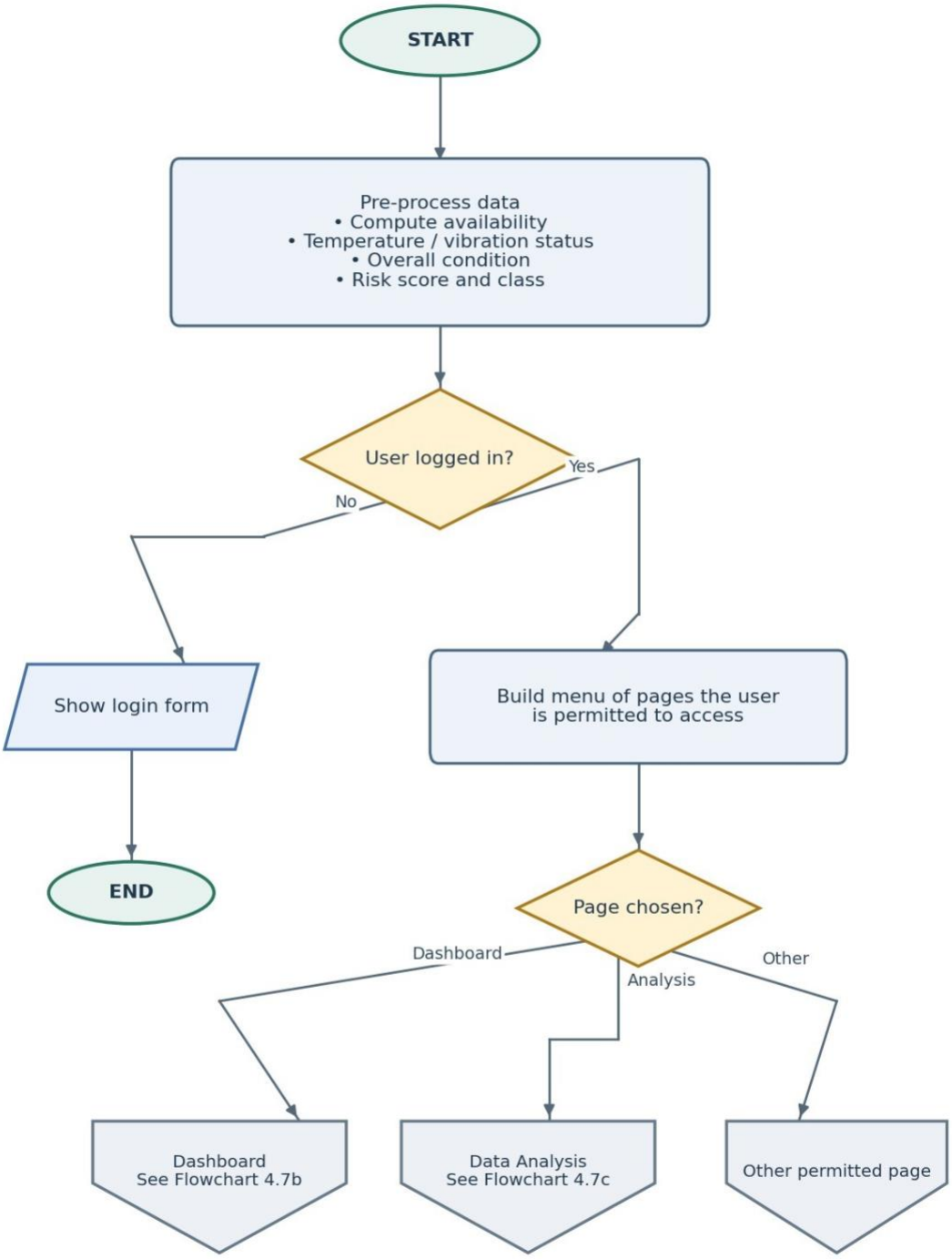
FLOWCHART 4.6D — RISK ALERT RULES

For each risk record



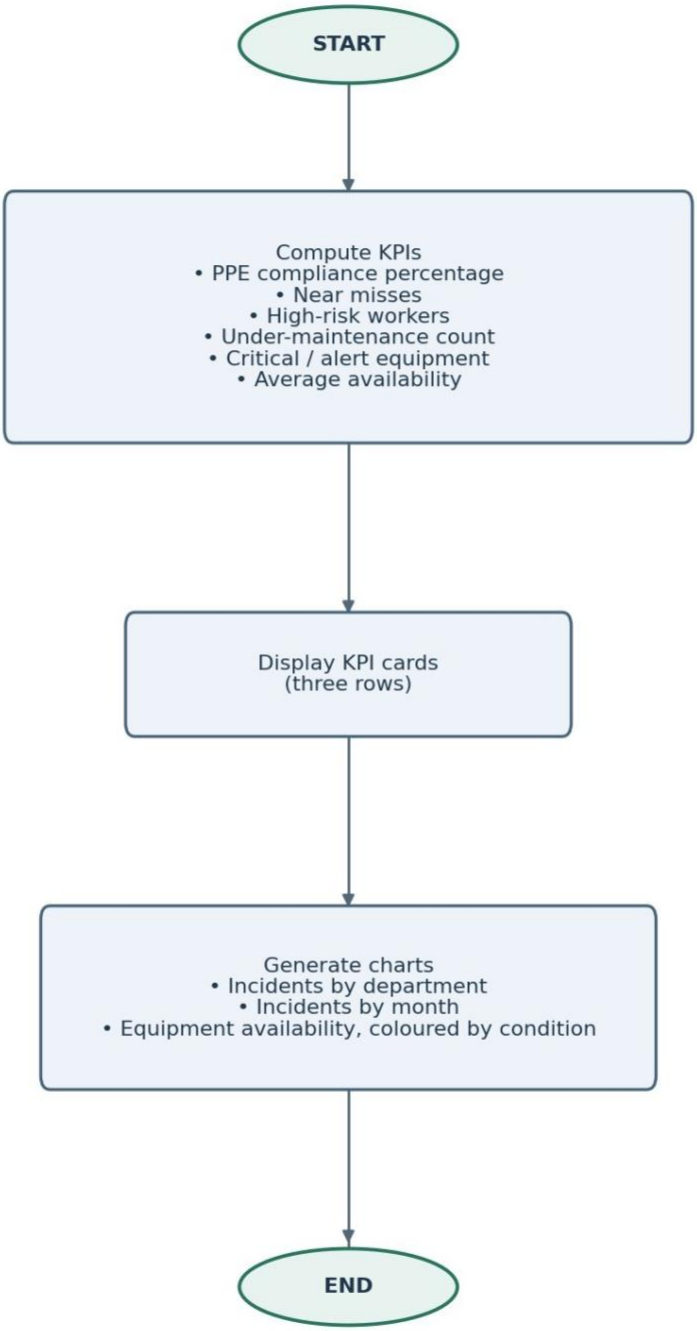
FLOWCHART 4.7a — DASHBOARD & DATA ANALYSIS

Main menu and access-control flow



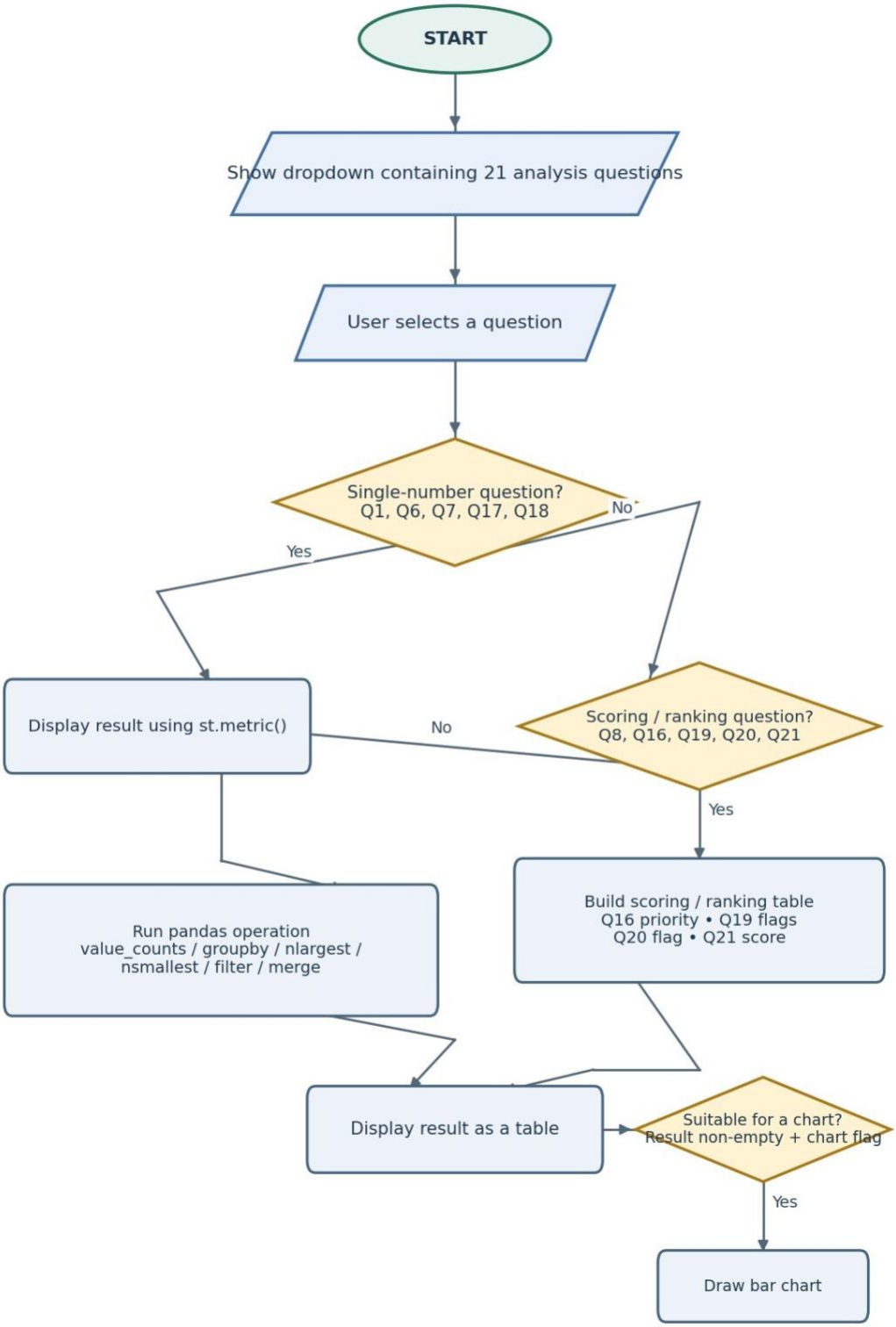
FLOWCHART 4.7b — DASHBOARD COMPUTATION

KPI cards and charts



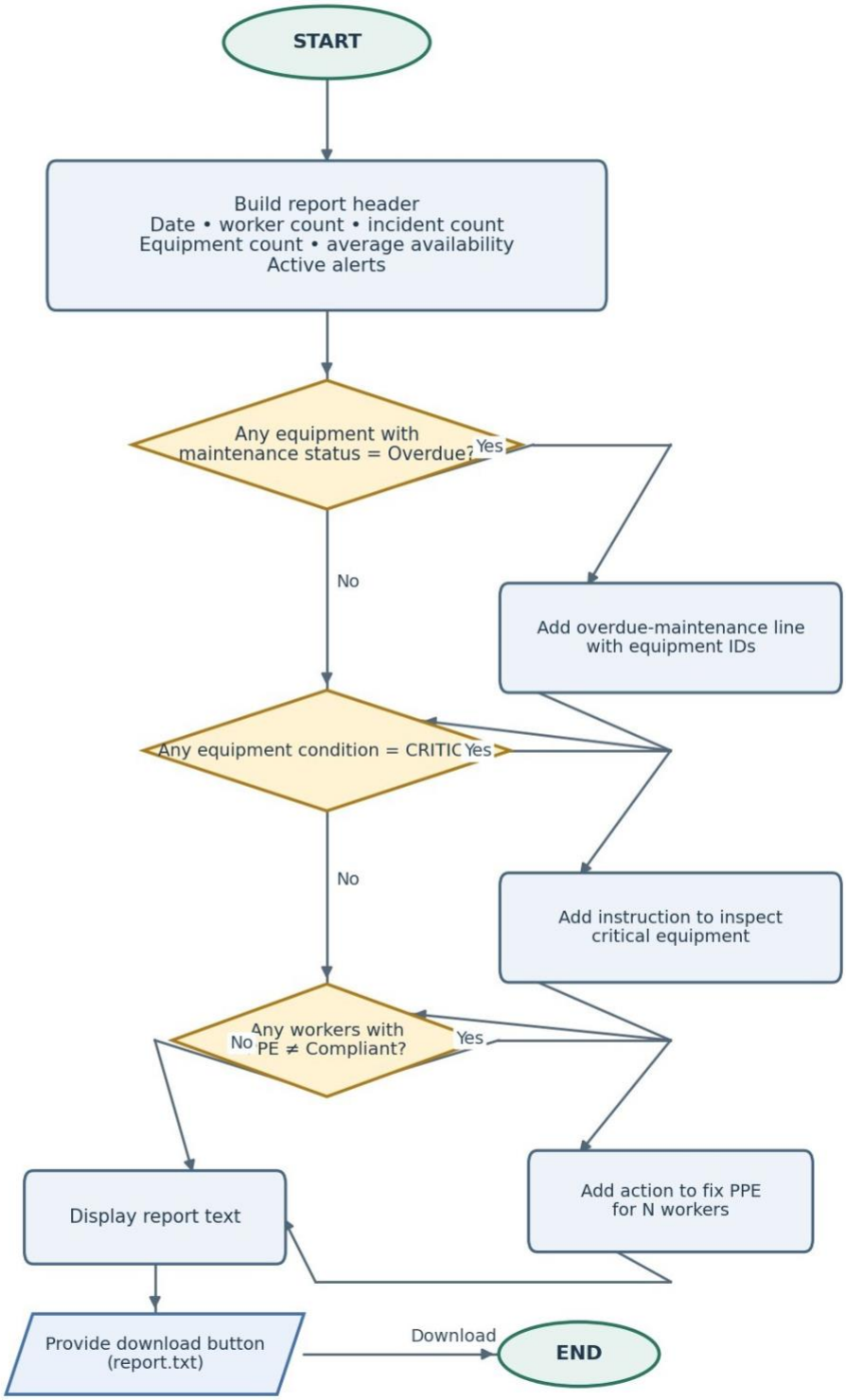
FLOWCHART 4.7c — DATA ANALYSIS

Selection and rendering of 21 analysis questions



FLOWCHART 4.8 — REPORTS

Report generation and downloadable output



DOCUMENT 2 — PSEUDOCODE

2.1 Helper Functions (used by all sections)

text

```
FUNCTION get_temp_status(temp):
    IF temp > 100 THEN RETURN "CRITICAL"
    ELSE IF temp >= 80 THEN RETURN "WARNING"
    ELSE RETURN "NORMAL"

FUNCTION get_vib_status(vib):
    IF vib > 8 THEN RETURN "CRITICAL"
    ELSE IF vib >= 5 THEN RETURN "WARNING"
    ELSE RETURN "NORMAL"

FUNCTION get_availability(operating_hrs, downtime_hrs):
    RETURN operating_hrs / (operating_hrs + downtime_hrs) * 100

FUNCTION get_risk_class(score):
    IF score <= 4 THEN RETURN "Low"
    ELSE IF score <= 9 THEN RETURN "Medium"
    ELSE IF score <= 16 THEN RETURN "High"
    ELSE RETURN "Critical"

FUNCTION count_table(column, label):
    counts ← count occurrences of each value in column
    RETURN table with columns [label, "Count"]

FUNCTION next_id(existing_ids, prefix, digits):
    max_num ← 0
```

```
    FOR EACH id IN existing_ids:
        num ← integer part of id after prefix
        IF num > max_num THEN max_num ← num
    RETURN prefix + zero-padded (max_num + 1) to `digits` digits

FUNCTION can_access(role, function_name):
    LOOKUP permission row where role=role AND function=function_name
    RETURN (row exists AND row.allowed = "Yes")
```

2.2 Pre-processing (run once at start-up)

```
text
LOAD users, permissions, workers, incidents, equipment,
     equipment_readings, equipment_alerts, maintenance_log,
     risk_assessments

FOR EACH equipment row:
    row.availability   ← get_availability(operating_hrs, downtime_hrs)
    row.temp_status    ← get_temp_status(temperature_c)
    row.vib_status     ← get_vib_status(vibration_mm_s)
    IF row.temp_status = "CRITICAL" OR row.vib_status = "CRITICAL" THEN
        row.condition ← "CRITICAL"
    ELSE IF row.temp_status = "WARNING" OR row.vib_status = "WARNING" THEN
        row.condition ← "WARNING"
    ELSE
        row.condition ← "NORMAL"

FOR EACH risk row:
    row.risk_score     ← likelihood × consequence
    row.classification ← get_risk_class(row.risk_score)
```

2.3 Alerts and Notifications (Section 4.6)

```
text
FUNCTION make_alerts():
    alerts ← empty list

    /* Equipment checks */
    FOR EACH eq IN equipment:
        IF eq.temp_status ≠ "NORMAL" THEN
            ADD [eq.temp_status, eq.equipment_id,
                "High temperature: " + eq.temperature_c + " C"]
        IF eq.vib_status ≠ "NORMAL" THEN
            ADD [eq.vib_status, eq.equipment_id,
                "High vibration: " + eq.vibration_mm_s + " mm/s"]
        IF eq.maintenance_status = "Overdue" THEN
            ADD ["WARNING", eq.equipment_id, "Overdue maintenance"]
        IF eq.downtime_hrs > 800 THEN
            ADD ["WARNING", eq.equipment_id,
                "Excessive downtime: " + eq.downtime_hrs + " h"]

    /* Worker checks */
    FOR EACH w IN workers:
        IF w.ppe_compliance ≠ "Compliant" THEN
            ADD ["WARNING", w.worker_id, "Low PPE compliance"]
        IF w.fatigue_level >= 8 THEN
            ADD ["WARNING", w.worker_id,
                "High fatigue level: " + w.fatigue_level]
        IF w.risk_level = "Critical" THEN
            ADD ["CRITICAL", w.worker_id,
                "High-risk safety observation"]

    /* Incident checks */
    FOR EACH inc IN incidents:
```

```
    IF inc.severity = "Critical" AND inc.incident_status ≠ "Closed" THEN
      ADD ["CRITICAL", inc.incident_id,
          "Critical safety incident: " + inc.incident_type]

  /* Risk checks */
  FOR EACH r IN risks:
    IF r.risk_score >= 17 THEN
      ADD ["CRITICAL", r.risk_id, "Critical risk: " + r.hazard]
    ELSE IF r.risk_score >= 10 THEN
      ADD ["WARNING", r.risk_id, "High risk: " + r.hazard]

  RETURN alerts SORTED BY severity



PROCEDURE display_alerts_page():
  alerts_now ← make_alerts()
  PRINT "Active alerts: " + LENGTH(alerts_now)
  SHOW table(alerts_now)

  PRINT "Equipment alert notices"
  FOR EACH eq IN equipment:
    IF eq.vib_status ≠ "NORMAL":
      PRINT alert notice (vibration, value, recommended action)
    IF eq.temp_status ≠ "NORMAL":
      PRINT alert notice (temperature, value, recommended action)
```

2.4 Dashboard (Section 4.7, part i – overview)

```
text
PROCEDURE display_dashboard():
  ppe_percent   ← % of workers with ppe_compliance = "Compliant"
  near_misses   ← count of incidents where incident_type = "Near Miss"
```

```
high_risk     ← count of workers where risk_level IN {"High","Critical"}
under_maint   ← count of equipment where maintenance_status = "Under Maintenance"
critical_eq   ← count of equipment where condition = "CRITICAL"
alert_eq      ← count of equipment where condition ≠ "NORMAL"

/* Row 1 – people & incidents */
SHOW metric "Total workers"            = LENGTH(workers)
SHOW metric "Safety incidents"         = LENGTH(incidents)
SHOW metric "Near misses"              = near_misses
SHOW metric "High-risk observations"   = high_risk

/* Row 2 – PPE & equipment */
SHOW metric "PPE compliance (%)"       = ROUND(ppe_percent, 1)
SHOW metric "Total equipment"          = LENGTH(equipment)
SHOW metric "Equipment available"      = LENGTH(equipment) − under_maint
SHOW metric "Under maintenance"        = under_maint

/* Row 3 – alerts & availability */
SHOW metric "Equipment alerts"         = alert_eq
SHOW metric "Critical equipment alerts" = critical_eq
SHOW metric "Average availability (%)"  = ROUND(MEAN(equipment.availability), 1)

/* Charts */
DRAW bar chart: incidents by department
DRAW line chart: incidents per month  (derived from date[:7])
DRAW bar chart: availability per equipment, coloured by condition
```

2.5 Data Analysis – (Section 4.7, part i – analysis)

```
text
PROCEDURE display_analysis():
   precompute:
```

```
    risk_by_dept = MEAN risk_score groupby department
    downtime     = SUM  downtime_hrs groupby department

question ← user selection from list of 21 questions
result   ← NULL
draw_chart← TRUE

SWITCH question:

CASE 1  "How many incidents occurred?"
    SHOW metric "Total incidents" = LENGTH(incidents)

CASE 2  "Which department has the most incidents?"
    result ← count_table(incidents.department)

CASE 3  "Which shift has the most incidents?"
    result ← count_table(incidents.shift)

CASE 4  "Which incident types are most common?"
    result ← count_table(incidents.incident_type)

CASE 5  "What are the most common causes?"
    result ← count_table(incidents.cause)

CASE 6  "How many incidents are Major or Critical?"
    big ← filter incidents WHERE severity IN {"Major","Critical"}
    SHOW metric "Major or Critical incidents" = LENGTH(big)
    result ← count_table(big.severity)

CASE 7  "What percentage of workers comply with PPE?"
    SHOW metric "PPE compliance (%)" = ROUND(ppe%, 1)
```

```
CASE 8  "Which departments have the highest safety risk?"
   result ← risk_by_dept SORTED BY avg_risk_score DESC

CASE 9  "Which equipment has the highest downtime?"
   result ← top 10 equipment BY downtime_hrs DESC

CASE 10 "Which equipment has the lowest availability?"
   result ← bottom 10 equipment BY availability ASC

CASE 11 "Which equipment has the highest vibration?"
   result ← top 10 equipment BY vibration_mm_s DESC

CASE 12 "Which equipment has the highest temperature?"
   result ← top 10 equipment BY temperature_c DESC

CASE 13 "Which equipment has the most alerts?"
   result ← top 10 equipment BY alert count (alerts_log) DESC

CASE 14 "Which equipment type has the highest downtime?"
   result ← SUM downtime_hrs groupby equipment_type, sorted DESC

CASE 15 "Which equipment requires immediate inspection?"
   result ← equipment WHERE condition = "CRITICAL"
          (show id, temperature, vibration)

CASE 16 "Which equipment should receive maintenance priority?"
   FOR EACH eq:
      points ← 0
      IF maintenance_status = "Overdue"           THEN points += 3
      IF maintenance_status = "Due for Maintenance" THEN points += 2
```

```
    IF condition = "CRITICAL"          THEN points += 3
    ELSE IF condition = "WARNING"      THEN points += 1
    IF availability < 85               THEN points += 1
    IF alert_count_30d >= 10           THEN points += 1
    store [eq.equipment_id, points]
  result ← top 10 sorted BY points DESC

CASE 17 "What percentage of equipment is operating normally?"
  pct ← % of equipment WHERE condition = "NORMAL"
  SHOW metric "Operating normally (%)" = ROUND(pct, 1)

CASE 18 "How many critical alerts are currently active?"
  critical_alerts ← filter make_alerts() WHERE severity = "CRITICAL"
  SHOW metric "Critical alerts active" = LENGTH(critical_alerts)
  result ← critical_alerts
  draw_chart ← FALSE

CASE 19 "Which safety indicators require management intervention?"
  indicators ← list of [name, value, target, flag] where flag =
          "ACTION NEEDED" if value outside target, else "OK":
    PPE compliance %       (target ≥ 90)
    Workers with fatigue ≥8 (target ≤ 3)
    Workers with expired training (target = 0)
    Open critical incidents (target = 0)
    Lost-time injuries     (target ≤ 10)
  result ← table(indicators)
  draw_chart ← FALSE

CASE 20 "Which departments have both high safety risk and high downtime?"
  result ← merge(risk_by_dept, downtime, ON department)
  risk_mean ← MEAN(result.avg_risk_score)
```

```
    down_mean ← MEAN(result.total_downtime)
    flagged   ← rows where BOTH scores > their respective means
    PRINT "Above average on both: " + list(flagged.departments)

  CASE 21 "Which areas require the greatest attention?"
    merge incidents-by-dept, risk_by_dept, downtime
    attention_points ← rank(incidents) + rank(avg_risk_score) + rank(downtime)
    result ← rows sorted BY attention_points DESC

  END SWITCH

  IF result ≠ NULL THEN
    SHOW table(result)
    IF draw_chart = TRUE AND LENGTH(result) > 0 THEN
      DRAW bar chart with result.columns[0] on X, columns[1] on Y,
        title = question
```

2.6 Reports (Section 4.8)

```
text
PROCEDURE generate_report():
  lines ← [
    "MINING SAFETY SUMMARY - " + TODAY,
    "Workers: "   + LENGTH(workers),
    "Incidents: " + LENGTH(incidents),
    "Equipment: " + LENGTH(equipment) +
      ", average availability " + ROUND(MEAN(availability),1) + "%",
    "Active alerts: " + LENGTH(make_alerts()),
    "",
    "RECOMMENDATIONS"
  ]
```

```
overdue  ← equipment WHERE maintenance_status = "Overdue"
IF LENGTH(overdue) > 0 THEN
    APPEND "- Do overdue maintenance on: " + join(overdue.ids)

critical ← equipment WHERE condition = "CRITICAL"
IF LENGTH(critical) > 0 THEN
    APPEND "- Inspect critical equipment now: " + join(critical.ids)

bad_ppe  ← workers WHERE ppe_compliance ≠ "Compliant"
IF LENGTH(bad_ppe) > 0 THEN
    APPEND "- Fix PPE problems for " + LENGTH(bad_ppe) + " workers"

report ← join(lines, newline)
DISPLAY report in text area
PROVIDE download button that saves report as “report.txt”
```